

DAY 12

NUMPY ADVANCED OPERATIONS

60-Day Data Science Learning Journey • Muhammad Ahmad

Purpose of this PDF: This version is designed as a complete beginner-friendly study document. Every important code example is explained line by line, including what the input means, what Python/NumPy does internally at a practical level, and what output you should expect.

Prerequisite: Day 11 NumPy Fundamentals. You should know arrays, shape, size, ndim, indexing, slicing and basic NumPy statistics.

What You Will Learn

- Understand NumPy axes without confusion.
- Calculate row-wise and column-wise statistics.
- Understand broadcasting with simple examples.
- Filter arrays with Boolean masks and multiple conditions.
- Use `np.where()` for decisions and classification.
- Sort data and find the indexes behind a ranking.
- Count unique values.
- Combine and split arrays.
- Understand the difference between copy and view.
- Use transpose and matrix operations.
- Build a complete Student Performance Analyzer.
- Read code line by line and understand why every line exists.

1. First: What Is an Axis?

Think about a table. Rows usually represent students and columns represent subjects. In NumPy, axis tells the direction in which a calculation is performed.

axis=0: move downward through rows, giving one result for each column.

axis=1: move horizontally across columns, giving one result for each row.

Example 1: Understanding the array

```
import numpy as np

marks = np.array([
    [80, 70, 90],
    [75, 85, 95]
])

print(marks)
```

Line-by-line:

`import numpy as np` → imports NumPy and gives it the short name `np`. `marks = np.array(...)` → creates a NumPy 2D array. First inner list `[80,70,90]` → marks for student 1. Second inner list `[75,85,95]` → marks for student 2. `print(marks)` → displays the complete array.

Expected output:

```
[[80 70 90]
 [75 85 95]]
```

Example 2: Total of everything, every subject, and every student

```
print(np.sum(marks))
print(np.sum(marks, axis=0))
print(np.sum(marks, axis=1))
```

Line-by-line:

`np.sum(marks)` → adds every value in the whole array. `np.sum(marks, axis=0)` → adds vertically. Column 1: 80+75, column 2: 70+85, column 3: 90+95. `np.sum(marks, axis=1)` → adds horizontally. Row 1: 80+70+90. Row 2: 75+85+95.

Expected output:

```
495
[155 155 185]
[240 255]
```

Memory trick: `axis=0` gives a result for each column. `axis=1` gives a result for each row.

2. Row and Column Averages

Student and subject averages

```
marks = np.array([
    [80, 75, 90],
    [70, 85, 80],
    [95, 90, 88]
])

student_average = np.mean(marks, axis=1)
subject_average = np.mean(marks, axis=0)

print("Student averages:", student_average)
print("Subject averages:", subject_average)
```

Line-by-line:

`marks = ...` → creates 3 students × 3 subjects. `np.mean(marks, axis=1)` → calculates one average for each row/student. `np.mean(marks, axis=0)` → calculates one average for each column/subject. The two results answer different questions.

Expected output:

```
Student averages: [81.66666667 78.33333333 91.        ]
Subject averages: [81.66666667 83.33333333 86.        ]
```

Maximum and minimum by subject

```
print(np.max(marks, axis=0))
print(np.min(marks, axis=0))
```

Line-by-line:

`np.max(..., axis=0)` → finds the highest mark in each subject column. `np.min(..., axis=0)` → finds the lowest mark in each subject column.

Expected output:

```
[95 90 90]
[70 75 80]
```

3. Broadcasting

Broadcasting means NumPy can apply a scalar or compatible smaller array across a larger array automatically.

Add the same bonus to every mark

```
marks = np.array([80, 75, 90])

result = marks + 5

print(result)
```

Line-by-line:

`marks = np.array(...)` → creates three marks. `result = marks + 5` → NumPy adds 5 to every element. You do not need a loop. `print(result)` → displays the new values.

Expected output:

```
[85 80 95]
```

Broadcasting a scalar over a 2D array

```
marks = np.array([
    [80, 75, 90],
    [70, 85, 80]
])

result = marks + 5
print(result)
```

Line-by-line:

marks → two rows of marks. marks + 5 → the single number 5 is broadcast to every element. result → contains every original mark plus 5.

Expected output:

```
[[85 80 95]
 [75 90 85]]
```

Broadcasting a subject-specific bonus

```
bonus = np.array([5, 2, 3])
result = marks + bonus
print(result)
```

Line-by-line:

bonus has three values because there are three subject columns. 5 is added to column 1 for every student. 2 is added to column 2 for every student. 3 is added to column 3 for every student.

Expected output:

```
[[85 77 93]
 [75 87 83]]
```

4. Comparisons and Boolean Masks

A Boolean mask is an array of True and False values. It can be used to filter the original array.

Create a condition

```
marks = np.array([45, 60, 75, 90])
mask = marks >= 70
print(mask)
print(marks[mask])
```

Line-by-line:

marks >= 70 → checks every value separately. mask → stores True for values that meet the condition and False for values that do not. marks[mask] → keeps only positions where mask is True.

Expected output:

```
[False False  True  True]
[75 90]
```

5. Multiple Conditions

AND condition with &

```
marks = np.array([45, 60, 75, 90, 95])
result = marks[(marks >= 60) & (marks <= 90)]
print(result)
```

Line-by-line:

marks >= 60 → first condition. marks <= 90 → second condition. & → requires both conditions to be True. marks[...] → returns only values satisfying both conditions.

Expected output:

```
[60 75 90]
```

Important: In NumPy, use & for element-wise AND and | for element-wise OR. Put each comparison inside parentheses.

OR condition with |

```
result = marks[(marks < 50) | (marks > 90)]
print(result)
```

Line-by-line:

marks < 50 → values below 50. marks > 90 → values above 90. | → keeps values satisfying either condition.

Expected output:

```
[45 95]
```

6. np.where(): Making Decisions

`np.where(condition, value_if_true, value_if_false)` is useful for creating labels such as Pass/Fail or categories such as A/B/C.

Pass or Fail

```
marks = np.array([45, 60, 75, 90])

status = np.where(marks >= 50, "Pass", "Fail")

print(status)
```

Line-by-line:

`marks >= 50` → creates the condition. "Pass" → selected when the condition is True. "Fail" → selected when the condition is False.
`status` → stores the final labels.

Expected output:

```
['Fail' 'Pass' 'Pass' 'Pass']
```

Nested np.where for grades

```
marks = np.array([92, 78, 65, 45])

grades = np.where(
    marks >= 80,
    "A",
    np.where(marks >= 60, "B", "C")
)

print(grades)
```

Line-by-line:

Outer condition: `marks >= 80`. If True, return A. If the outer condition is False, NumPy evaluates the inner `np.where`. Inner condition: `marks >= 60`. If True, return B. Otherwise return C.

Expected output:

```
['A' 'B' 'B' 'C']
```

7. Sorting and argsort()

Sort values

```
marks = np.array([85, 65, 95, 72, 90])

ascending = np.sort(marks)
descending = ascending[::-1]

print(ascending)
print(descending)
```

Line-by-line:

`np.sort(marks)` → creates sorted values from smallest to largest. `[::-1]` → reverses the sorted array. `ascending` → keeps the low-to-high order. `descending` → keeps the high-to-low order.

Expected output:

```
[65 72 85 90 95]
[95 90 85 72 65]
```

Find ranking indexes with argsort()

```
indexes = np.argsort(marks)
print(indexes)
```

Line-by-line:

`np.argsort(marks)` does not return the sorted marks. It returns the original positions that would produce sorted order. For example, 65 was at index 1, so 1 appears first.

Expected output:

```
[1 3 0 4 2]
```

8. Unique Values and Counts

Count grades

```
grades = np.array(["A", "B", "A", "C", "B", "A"])

values, counts = np.unique(
    grades,
    return_counts=True
)

print(values)
print(counts)
```

Line-by-line:

grades → stores repeated category values. np.unique(grades) → finds each different value once. return_counts=True → also asks NumPy to count each unique value. values → unique grade labels. counts → number of occurrences for each corresponding label.

Expected output:

```
['A' 'B' 'C']
[3 2 1]
```

9. Combining Arrays

Concatenate

```
a = np.array([10, 20, 30])
b = np.array([40, 50, 60])

result = np.concatenate((a, b))
print(result)
```

Line-by-line:

a and b → two 1D arrays. np.concatenate((a,b)) → joins them end-to-end. result → contains six values.

Expected output:

```
[10 20 30 40 50 60]
```

Vertical and horizontal stacking

```
print(np.vstack((a, b)))
print(np.hstack((a, b)))
```

Line-by-line:

vstack → places a and b as separate rows. hstack → joins the values horizontally into one longer 1D array.

Expected output:

```
[[10 20 30]
 [40 50 60]]
[10 20 30 40 50 60]
```

10. Splitting Arrays

Split into equal parts

```
arr = np.array([10, 20, 30, 40, 50, 60])

parts = np.split(arr, 3)

print(parts)
```

Line-by-line:

arr → six values. np.split(arr, 3) → divides six values into three equal sections. Each section contains two values. parts → stores the three resulting arrays.

Expected output:

```
[array([10, 20]), array([30, 40]), array([50, 60])]
```

11. Copy vs View

This is important because accidentally modifying shared data can create difficult bugs.

Independent copy

```
arr = np.array([10, 20, 30])

copy_arr = arr.copy()
copy_arr[0] = 100

print("Original:", arr)
print("Copy:", copy_arr)
```

Line-by-line:

arr.copy() → creates an independent array. copy_arr[0] = 100 → changes only the copy. The original arr stays unchanged.

Expected output:

```
Original: [10 20 30]
Copy: [100 20 30]
```

View

```
arr = np.array([10, 20, 30])
view_arr = arr.view()
```

```
view_arr[0] = 100

print("Original:", arr)
print("View:", view_arr)
```

Line-by-line:

`arr.view()` → creates a view that can share the same underlying data. Changing `view_arr[0]` can therefore change `arr[0]` too. Use `copy()` when you need an independent array.

Expected output:

```
Original: [100 20 30]
View: [100 20 30]
```

12. Insert and Delete

Insert a value

```
arr = np.array([10, 20, 30])

new_arr = np.insert(arr, 1, 99)

print(new_arr)
```

Line-by-line:

`np.insert(arr, 1, 99)` → index 1 is the insertion position. 99 → the value to insert. The original array is not changed in place. A new array is returned.

Expected output:

```
[10 99 20 30]
```

Delete a value

```
arr = np.array([10, 20, 30, 40])

new_arr = np.delete(arr, 2)

print(new_arr)
```

Line-by-line:

Index 2 points to value 30. `np.delete` removes that position and returns a new array.

Expected output:

```
[10 20 40]
```

13. Transpose

Transpose a matrix

```
matrix = np.array([
    [1, 2, 3],
    [4, 5, 6]
])

transposed = matrix.T

print(transposed)
```

Line-by-line:

`matrix` has 2 rows and 3 columns. `matrix.T` means transpose. Rows become columns and columns become rows. The resulting shape changes from (2,3) to (3,2).

Expected output:

```
[[1 4]
 [2 5]
 [3 6]]
```

14. Matrix Operations

Element-wise vs matrix multiplication

```
A = np.array([
    [1, 2],
    [3, 4]
])

B = np.array([
    [5, 6],
    [7, 8]
])
```

```

print("Element-wise:")
print(A * B)

print("Matrix multiplication:")
print(A @ B)

```

Line-by-line:

A and B → two 2×2 matrices. A * B → multiplies matching positions: 1*5, 2*6, 3*7, 4*8. A @ B → performs mathematical matrix multiplication. The @ operator is commonly used for matrix multiplication in Python.

Expected output:

```

Element-wise:
[[ 5 12]
 [21 32]]

Matrix multiplication:
[[19 22]
 [43 50]]

```

15. Practical Data Science Example

Student analysis with NumPy

```

students = np.array([
    [85, 78, 90],
    [72, 88, 75],
    [95, 91, 89],
    [65, 70, 68]
])

student_avg = np.mean(students, axis=1)
subject_avg = np.mean(students, axis=0)

high_students = students[student_avg >= 80]

print("Student averages:", student_avg)
print("Subject averages:", subject_avg)
print("High-performing students:")
print(high_students)

```

Line-by-line:

students → rows are students and columns are subjects. student_avg → one average per row. subject_avg → one average per column. student_avg >= 80 → Boolean mask. students[...] → filters complete student rows using that mask.

Expected output:

```

Student averages: [84.33333333 78.33333333 91.66666667 67.66666667]
Subject averages: [79.25 81.75 80.5 ]
High-performing students:
[[85 78 90]
 [95 91 89]]

```

16. Practice Tasks: Do These Yourself First

Task 1

Create a 3×3 marks array. Calculate each student's total and average and each subject's average.

Task 2

Using the same array, calculate maximum and minimum for each subject.

Task 3

For [45,55,67,72,81,89,92,38,96], filter >=50, >=80 and 60-90.

Task 4

Use np.where() for Pass/Fail with 50 as the passing mark.

Task 5

Create grades: 90+ A+, 80-89 A, 70-79 B, 60-69 C, below 60 F.

Task 6

Sort [85,67,95,72,88,91] ascending and descending and find argsort indexes.

Task 7

Find unique values and counts in ['A','B','A','C','B','A','D','C'].

Task 8

Combine [10,20,30] and [40,50,60] using concatenate, vstack and hstack.

Task 9

Transpose [[1,2,3],[4,5,6]], show shape and multiply by 2.

Task 10

Build the complete Student Performance Analyzer described in the next section.

17. Full Solutions

Solution 1 and 2: Row/Column Analysis

Complete solution

```
import numpy as np

marks = np.array([
    [80, 70, 90],
    [75, 85, 95],
    [90, 88, 92]
])

student_total = np.sum(marks, axis=1)
student_average = np.mean(marks, axis=1)
subject_average = np.mean(marks, axis=0)
subject_max = np.max(marks, axis=0)
subject_min = np.min(marks, axis=0)

print("Student totals:", student_total)
print("Student averages:", student_average)
print("Subject averages:", subject_average)
print("Subject max:", subject_max)
print("Subject min:", subject_min)
```

Line-by-line:

The first five lines create the data. sum axis=1 → totals by student. mean axis=1 → averages by student. mean axis=0 → averages by subject. max/min axis=0 → maximum/minimum for each subject.

Expected output:

```
Student totals: [240 255 270]
Student averages: [80. 85. 90.]
Subject averages: [81.66666667 81.          92.33333333]
Subject max: [90 88 95]
Subject min: [75 70 90]
```

Solution 3: Boolean Filtering

Complete solution

```
marks = np.array([45, 55, 67, 72, 81, 89, 92, 38, 96])

print(marks[marks >= 50])
print(marks[marks >= 80])
print(marks[(marks >= 60) & (marks <= 90)])
```

Line-by-line:

Each condition creates a Boolean mask. Indexing with that mask returns only matching values.

Expected output:

```
[55 67 72 81 89 92 96]
[81 89 92 96]
[67 72 81 89]
```

Solution 4: Pass/Fail

Complete solution

```
status = np.where(marks >= 50, "Pass", "Fail")
```

```
print(status)
```

Line-by-line:

For every mark, NumPy checks whether it is at least 50 and selects the correct label.

Expected output:

```
['Fail' 'Pass' 'Pass' 'Pass' 'Pass' 'Pass' 'Pass' 'Fail' 'Pass']
```

Solution 5: Grade Classification

Complete solution

```
grades = np.where(
    marks >= 90, "A+",
    np.where(
        marks >= 80, "A",
        np.where(
            marks >= 70, "B",
            np.where(marks >= 60, "C", "F")
        )
    )
)

print(grades)
```

Line-by-line:

NumPy checks from highest boundary to lowest. 90 or above → A+. Otherwise 80 or above → A. Otherwise 70 or above → B. Otherwise 60 or above → C. Anything below 60 → F.

Expected output:

```
['F' 'F' 'B' 'B' 'A' 'A' 'A+' 'F' 'A+']
```

Solution 6: Sorting

Complete solution

```
marks = np.array([85, 67, 95, 72, 88, 91])

print("Ascending:", np.sort(marks))
print("Descending:", np.sort(marks)[::-1])
print("Indexes:", np.argsort(marks))
```

Line-by-line:

sort gives values in order. [::-1] reverses the sorted result. argsort gives original positions in sorted order.

Expected output:

```
Ascending: [67 72 85 88 91 95]
Descending: [95 91 88 85 72 67]
Indexes: [1 3 0 4 5 2]
```

Solution 7: Unique and Counts

Complete solution

```
grades = np.array(["A", "B", "A", "C", "B", "A", "D", "C"])

values, counts = np.unique(grades, return_counts=True)

print("Values:", values)
print("Counts:", counts)
```

Line-by-line:

np.unique identifies categories. return_counts=True gives how many times each category occurs.

Expected output:

```
Values: ['A' 'B' 'C' 'D']
Counts: [3 2 2 1]
```

Solution 8: Combining Arrays

Complete solution

```
a = np.array([10, 20, 30])
b = np.array([40, 50, 60])

print(np.concatenate((a, b)))
print(np.vstack((a, b)))
print(np.hstack((a, b)))
```

Line-by-line:

concatenate joins the arrays. vstack creates separate rows. hstack joins them horizontally.

Expected output:

```
[10 20 30 40 50 60]
[[10 20 30]
 [40 50 60]]
[10 20 30 40 50 60]
```

Solution 9: Matrix Practice

Complete solution

```
matrix = np.array([
    [1, 2, 3],
    [4, 5, 6]
])

print("Transpose:")
print(matrix.T)

print("Shape:", matrix.shape)

print("Times 2:")
print(matrix * 2)
```

Line-by-line:

matrix.T changes rows into columns. matrix.shape tells us rows and columns. matrix * 2 broadcasts the scalar 2 over every element.

Expected output:

```
Transpose:
[[1 4]
 [2 5]
 [3 6]]
Shape: (2, 3)
Times 2:
[[ 2  4  6]
 [ 8 10 12]]
```

18. Mini Project: Student Performance Analyzer

Project requirements: Math, English and CS are the three columns. For five students, calculate student averages, subject averages, subject maximum/minimum, students with average ≥ 80 , students with average ≥ 70 , pass/fail using average ≥ 50 , class average, highest student average, and grades.

Complete project

```
import numpy as np

students = np.array([
    [85, 78, 90],
    [72, 88, 75],
    [95, 91, 89],
    [65, 70, 68],
    [55, 62, 58]
])

student_avg = np.mean(students, axis=1)
subject_avg = np.mean(students, axis=0)
subject_max = np.max(students, axis=0)
subject_min = np.min(students, axis=0)

print("Student averages:", student_avg)
print("Subject averages:", subject_avg)
print("Subject maximums:", subject_max)
print("Subject minimums:", subject_min)

high_80 = students[student_avg >= 80]
high_70 = students[student_avg >= 70]

print("Students avg >= 80:")
print(high_80)

print("Students avg >= 70:")
print(high_70)

status = np.where(student_avg >= 50, "Pass", "Fail")
print("Status:", status)

class_average = np.mean(students)
highest_average = np.max(student_avg)

print("Class average:", class_average)
print("Highest student average:", highest_average)
grades = np.where(
```

```

        student_avg >= 90, "A+",
        np.where(
            student_avg >= 80, "A",
            np.where(
                student_avg >= 70, "B",
                np.where(
                    student_avg >= 60, "C",
                    np.where(student_avg >= 50, "D", "F")
                )
            )
        )
    )
)

print("Grades: ", grades)

```

Line-by-line:

students → input dataset. student_avg → one average per student. subject_avg → one average per subject. subject_max/min → subject-level boundaries. student_avg >= 80 → Boolean mask used to select complete student rows. np.where(student_avg >= 50, ...) → creates Pass/Fail labels. np.mean(students) → average of all marks in the complete dataset. np.max(student_avg) → highest student's average. Nested np.where → converts numerical averages into letter grades.

Expected output:

```

Student averages: [84.33333333 78.33333333 91.66666667 67.66666667 58.33333333]
Subject averages: [74.4 77.8 76. ]
Subject maximums: [95 91 90]
Subject minimums: [55 62 58]

Students avg >= 80:
[[85 78 90]
 [95 91 89]]

Students avg >= 70:
[[85 78 90]
 [72 88 75]
 [95 91 89]]

Status: ['Pass' 'Pass' 'Pass' 'Pass' 'Pass']
Class average: 76.06666666666667
Highest student average: 91.66666666666667
Grades: ['A' 'B' 'A+' 'C' 'D']

```

19. Project: How the Data Flows

Step 1: Input marks → NumPy stores the numerical data efficiently.

Step 2: axis=1 → calculate information for each student.

Step 3: axis=0 → calculate information for each subject.

Step 4: Boolean masks → select students meeting performance conditions.

Step 5: np.where() → convert conditions into meaningful labels.

Step 6: Output → present statistics that can support further analysis or visualization.

This is the same general pattern used later with real datasets: **load** → **transform** → **filter** → **calculate** → **classify** → **analyze**.

20. Common Mistakes and Fixes

- **Axis confusion:** Remember axis=0 gives one result per column, axis=1 gives one result per row.
- **Wrong operators:** Use & and | for NumPy element-wise conditions.
- **Missing parentheses:** Write (marks >= 60) & (marks <= 90).
- **Wrong multiplication:** A*B is element-wise. A@B is matrix multiplication.
- **Unexpected changes:** A view may share data. Use copy() when you need independence.
- **Overusing loops:** Prefer NumPy vectorized operations when they express the calculation clearly.

21. Quick Revision Table

Topic	Remember this	Example
axis=0	One result per column	<code>np.mean(a,axis=0)</code>
axis=1	One result per row	<code>np.mean(a,axis=1)</code>
Broadcasting	Apply compatible values automatically	<code>a+5</code>
Mask	Filter by condition	<code>a[a>=70]</code>
where	Create conditional labels	<code>np.where(...)</code>
sort	Sort values	<code>np.sort(a)</code>
argsort	Get sorting positions	<code>np.argsort(a)</code>
unique	Unique values/counts	<code>np.unique(a)</code>
concatenate	Join arrays	<code>np.concatenate(...)</code>
vstack	Stack as rows	<code>np.vstack(...)</code>
hstack	Join horizontally	<code>np.hstack(...)</code>
copy	Independent array	<code>a.copy()</code>
view	May share data	<code>a.view()</code>
transpose	Rows ↔ columns	<code>a.T</code>
@	Matrix multiplication	<code>A@B</code>

22. Data Science Connection

NumPy is one of the numerical foundations of Python Data Science. The operations in this lesson appear in data preparation, exploratory data analysis, feature engineering, statistics and machine learning. Pandas will later provide labeled tables and DataFrames, while many scientific Python tools use NumPy underneath.

23. Final Learning Checklist

- Can I explain axis=0 in my own words?
- Can I explain axis=1 in my own words?
- Can I calculate student and subject averages?
- Can I use broadcasting?
- Can I create Boolean masks?
- Can I combine multiple conditions with & and |?
- Can I use np.where() for classification?
- Can I sort values and find argsort indexes?
- Can I count unique values?
- Can I combine and split arrays?
- Do I understand copy versus view?
- Can I transpose a matrix?
- Do I understand * versus @ for matrices?
- Can I build the Student Performance Analyzer without copying the solution?

24. What You Should Practice Before Day 13

Run every code example yourself. Then change the marks, add another student, add another subject, change the passing threshold, and create your own grade rules. The goal is not only to read the code but to understand what happens when you change the input.

DAY 12 COMPLETE

Next stage: Pandas and tabular DataFrame analysis.